Optimisations du jeu VR

Ce qui a été fait concrètement, où le trouver dans le projet, et comment le vérifier

Projet	SAÉ 5D.01 · tower defense VR (Quincy à l'arc, singes posés à la main) · BUT MMI 3, IUT Béziers
Branche	`feat/opti-fusion`, fusionnée dans `main` par la PR #188 le 10 nov. 2026
Issues	#174 à #187 (optimisations), #153 et #151 (branche d'optimisation et mode admin), #35 (mesure des fps)
Par	Dylan, avec Claude (Claude Code). Chaque usage de l'IA est noté dans `docs/journal/dylan.md`
Objectif	tenir 72 images par seconde sur le Meta Quest (règle de confort de la SAÉ)

À savoir avant de lire. Tout ce qui suit est dans le code et compile. Mais **rien n'a encore été mesuré au casque** : chaque fiche dit comment le vérifier, et la dernière page donne le tableau des mesures à remplir. Les chiffres « avant » connus viennent de l'issue #153 (mode admin, 200 singes) : **41 fps, CPU limitant, 231 skinned meshes, 626 000 triangles, 141 SetPass calls**.

Le détail de chaque optimisation (problème, avant / après, suggestions) est aussi dans `docs/opti.md`. Ce document en est le guide de vérification.

Sommaire

Partie	Contenu	Page
1. Les outils pour vérifier	Profiler, Frame Debugger, Stats, compteur de fps, mode admin, logcat	3
2. Rendu au casque	réglages mobiles, post-traitement, fovéation, distance de vue, build allégé	4
3. Modèles et textures	LOD, textures limitées, pluie	6
4. Ballons	instancing du plateau, pool, Rigidbody kinematic, taches du camo	7
5. Matériaux et ombres	matériaux partagés, ombres allégées	9
6. Décor	static et occlusion	9
7. Singes	figés, immobiles loin du joueur, ciblage, aura	10
8. Objets et sons	pools, préchargement, Update coupés	11
9. Corrections après relecture	bugs trouvés dans la PR et corrigés avant le merge	12
10. Pas fait, et pourquoi	ce qu'on a choisi de ne pas faire	13
11. Mesures à faire	la procédure et le tableau à remplir au casque	13

Comment lire une fiche

Chaque optimisation a la même fiche : **ce qui a été fait** (en clair), **où** (fichier et numéro de ligne dans `main`), **comment vérifier** (les clics à faire dans Unity ou au casque) et, s'il y en a, **ce qu'on perd** en visuel ou en jeu. Les chemins de scripts partent de `Unity/Assets/_Project/Scripts/`, noté `Scripts/`.

1. Les outils pour vérifier

Toujours mesurer de la même façon, au même endroit, pour comparer avant et après.

Outil	Où le trouver	Ce qu'on y lit
Compteur de fps	`Scripts/Player/FpsCounter.cs` au poignet gauche, en jeu	les images par seconde ; vert à partir de 72
Stats	bouton **Stats** en haut à droite de la vue Game, en Play	Batches, SetPass calls, Tris (triangles) de l'image
Profiler	Window → Analysis → Profiler	CPU (temps par image, GC Alloc), Rendering (Batches, SetPass), Memory (nombre de matériaux, textures), Physics
Frame Debugger	Window → Analysis → Frame Debugger → Enable	chaque appel de dessin de l'image, regroupé ou non (SRP Batch, instancing)
Mode admin	SAE → Mode de jeu → PC admin, puis F1 en Play	panneau de triche : poser 50-200 singes, aller à une vague, argent illimité ; affiche les fps
Occlusion	Window → Rendering → Occlusion Culling → Visualization	ce que la caméra dessine vraiment
logcat (casque)	`adb logcat -s Unity` avec le casque branché	la ligne « Rendu : qualité … asset … » au lancement
OVR Metrics Tool	appli Meta à installer sur le casque	fps, temps GPU et CPU, niveau de fovéation en surimpression

Les deux endroits de test

- **Hub** : dans la cabane, face à la bibliothèque.
- **Carte** : au point d'arrivée des ballons, pendant une vague (la même vague à chaque fois, par exemple via le mode admin).

Éditeur ≠ casque. Les chiffres de l'éditeur ne sont pas ceux du Quest (son GPU est celui d'un téléphone). L'éditeur dit si on va dans le bon sens ; seul le casque dit si on tient 72 fps. Plusieurs réglages ne s'appliquent **qu'au casque** (post-traitement, fovéation, pluie, textures) : dans l'éditeur, ne pas s'étonner que rien ne change.

2. Rendu au casque

1. Bon réglage de rendu au casque #186

Ce qui a été fait	Le niveau de qualité utilisé par le build Quest (« Meta Quest (Build Profile) ») pointait sur l'asset URP **par défaut du paquet**, et pas sur nos réglages mobiles. Il pointe maintenant sur `Mobile_RPAsset` : ombres à 15 m, réglages pensés pour le GPU du Quest.
Où	• `Unity/ProjectSettings/QualitySettings.asset` ligne 160 : `customRenderPipeline` = le même asset que le niveau « Mobile » (ligne 51) • `Unity/Assets/Settings/Mobile_RPAsset.asset` : l'asset de rendu mobile • `Scripts/Player/FpsCounter.cs:26` : écrit au lancement quelle qualité et quel asset sont utilisés
Comment vérifier	• Edit → Project Settings → **Quality** : cliquer sur « Meta Quest (Build Profile) » ; le champ **Render Pipeline Asset** doit être **Mobile_RPAsset**. • Au casque (build de développement), `adb logcat -s Unity` : chercher la ligne *Rendu : qualité « Meta Quest (Build Profile) », asset « Mobile_RPAsset ».*

2. Post-traitement coupé au casque #187

Ce qui a été fait	Le réglage des tons, des couleurs et de la balance des blancs oblige le Quest à dessiner l'image dans une texture à part, puis à la recopier en plein écran pour les deux yeux. C'est cher, et ça empêche la fovéation de marcher. Au casque (Android), la caméra ne fait plus de post-traitement ; sur PC, rien ne change.
Où	• `Scripts/Player/HeadsetRendering.cs` : `renderPostProcessing = false`, seulement si la plateforme est Android • le script est ajouté au joueur par `Scripts/Player/PlayerRig.cs` : rien à régler dans les scènes
Comment vérifier	• Ouvrir `HeadsetRendering.cs` : la ligne est après le test `RuntimePlatform.Android`. • Au casque : comparer à une capture d'avant ; l'image est un peu moins « étalonnée ». • Avec OVR Metrics Tool : le temps GPU doit baisser par rapport à un build d'avant le 9 nov.
Ce qu'on perd	un peu de chaleur des couleurs au casque.

3. Fovéation dynamique #175

Ce qui a été fait	Les bords de l'image, que l'œil voit flou, sont calculés en plus basse résolution. La fonction « Meta XR Foveation » était cochée dans OpenXR, mais aucun niveau n'était réglé : le code règle maintenant le niveau **High**, en mode **dynamique** (le casque ne l'active que quand une image risque d'être en retard).
Où	• `Scripts/Player/HeadsetRendering.cs` : `OVRManager.foveatedRenderingLevel = High` et `useDynamicFoveatedRendering = true`, une image après le démarrage (la session OpenXR doit être ouverte) • `Unity/Assets/XR/Settings/OpenXR Package Settings.asset` : la fonction de fovéation (Android)
Comment vérifier	• Edit → Project Settings → XR Plug-in Management → **OpenXR**, onglet Android : « Meta XR Foveation » cochée. • Au casque, avec OVR Metrics Tool : la ligne *Foveation Level* monte pendant une grosse vague. • Regarder les bords de l'image en tournant la tête pendant une vague : un léger flou sur les côtés est normal.
Ce qu'on perd	un peu de netteté sur les bords, seulement quand le casque est chargé.

4. Distance de vue réduite à 250 m #184	
Ce qui a été fait	La caméra dessinait jusqu'à 400 m dans les scènes (1 000 m dans le prefab XRI). La carte et le hub font quelques dizaines de mètres ; 250 m couvrent encore les nuages de l'orage (qui arrivent de 150 m) et la fin du brouillard (230 m).
Où	• `Scripts/Player/HeadsetRendering.cs:19` : `FarClip = 250f` ; appliqué ligne 25, sur PC comme au casque • `Scripts/Editor/PrototypeGenerator.cs:321`, 363 et 398 : les scènes générées utilisent la même valeur
Comment vérifier	• En Play, dans la Hierarchy, sélectionner la caméra du joueur (sous XR Origin) : **Clipping Planes → Far = 250**. • Regarder l'horizon sur la carte et pendant l'orage : rien ne doit être coupé.

5. Build allégé #183 (en partie)	
Ce qui a été fait	Les messages de log et d'avertissement n'enregistrent plus leur trace de pile (coûteux à chaque message) ; les erreurs la gardent. Le message de diagnostic du coffre n'est écrit que dans l'éditeur.
Où	• `Unity/ProjectSettings/ProjectSettings.asset:57` : `m_StackTraceTypes` • `Scripts/Runtime/ChestController.cs:152` : le `Debug.Log` du coffre, entouré d'un test « éditeur seulement »
Comment vérifier	• Edit → Project Settings → Player → Other Settings → **Stack Trace** : Log = None, Warning = None, Error = Script Only. • **Reste à faire** pour le rendu (#183, ouverte) : faire le build final **sans** « Development Build ».

3. Modèles et textures

6. LOD automatique sur les modèles #176

Ce qui a été fait	La case **Mesh LOD** de Unity 6 est cochée à l'import des modèles sans squelette. Unity calcule tout seul des versions simplifiées dans le même maillage, et la carte graphique prend la plus légère qui suffit selon la taille à l'écran. C'est important pour les ballons : un ballon fait 1 032 triangles (le cœur 1 680), et il y en a des centaines par vague.
Où	• 8 fichiers `.fbx.meta` avec `generateMeshLods: 1` : `Art/Ballons/FBX/Ballon_Normal`, `Ballon_Blindage`, `Ballon_Coeur`, `Art/MOAB/FBX/MOAB`, `Art/BFB/FBX/BFB`, `Art/Bananier/FBX/Bananier`, `Art/Canon/FBX/Canon`, `Art/Tireur/FBX/Tireur` • pas de réexport Blender : réversible en décochant la case
Comment vérifier	• Dans Project, sélectionner `Ballon_Normal.fbx` → Inspector, onglet **Model** : « Generate Mesh LODs » est coché. • En Play, sur la carte pendant une vague : ouvrir **Stats** et noter **Tris** près des ballons, puis loin. Ça doit baisser de loin.
Ce qu'on perd	rien de visible : les versions simplifiées ne servent que de loin.

7. Textures limitées sur le casque #176

Ce qui a été fait	Les modèles de Blender gardaient leur texture **dans** le FBX (d'où des singes de 700 triangles qui pèsent 1,9 Mo), et Unity ne laisse pas régler une texture intégrée. Le menu **SAE → Optimiser les textures (Quest)** les sort du FBX, puis limite sur Android toutes les textures du dossier Art à **1024 px** (512 pour les projectiles), avec mipmaps (des versions plus petites pour le lointain). Lancé le 10 nov. : **30 FBX vidés, 18 textures extraites, 39 textures limitées**.
Où	• `Scripts/Editor/TextureMenu.cs` : le menu • les textures extraites : `Art//Textures/Extraites/` (ex. `Art/Singe_Base/Textures/Extraites/SingeBase_Texture.png`), suivies par Git LFS • les réglages : dans le `.png.meta` de chaque texture (bloc Android, `maxTextureSize: 1024`)
Comment vérifier	• Sélectionner `SingeBase_Texture.png` (dans Extraites) → Inspector, onglet **Android** : « Override » coché, Max Size **1024**, Format Automatic (ASTC). Onglet par défaut : « Generate Mipmaps » coché. • Un projectile, par ex. `SingeBase_Dart_Texture.png` : Max Size **512**. • **À vérifier à l'œil** : les singes, Quincy et les ballons gardent leurs couleurs (aucun modèle blanc ou rose). • Window → Analysis → Memory Profiler, au casque : mémoire des textures avant / après.
Ce qu'on perd	un peu de netteté de très près au casque. Les mipmaps prennent un tiers de mémoire en plus sur PC (réglage commun à toutes les plateformes).

8. Pluie allégée au casque #180

Ce qui a été fait	Pendant l'orage des vagues de boss, la pluie pouvait faire 1 000 gouttes transparentes. Au casque, on n'en garde que 40 % (400 au plus), en traits 1,5 fois plus longs pour que la pluie paraisse aussi dense.
Où	• `Scripts/Map/Weather.cs:309` : `HeadsetRainShare = 0.4f` ; appliqué ligne 354 • `Scripts/Map/Weather.cs:308` : `Headset` = la plateforme est Android
Comment vérifier	• Au casque, pendant une vague de boss : la pluie reste visible et dense. • Dans l'éditeur, rien ne change (c'est normal).
Ce qu'on perd	un peu de densité de pluie au casque.

4. Ballons

9. Instancing des ballons du plateau #153

Ce qui a été fait	Le plateau du hub montre la carte en miniature, en direct. Avant, chaque ballon avait sa miniature (un objet de plus par ballon). Maintenant, les miniatures ne sont plus des objets : à chaque image, toutes les boules d'une même couleur sont dessinées en **un seul appel** (`Graphics.RenderMeshInstanced`). 500 ballons = quelques appels. **Sauf les boss et les dirigeables**, peu nombreux, qui gardent une vraie copie de leur modèle pour qu'on les reconnaisse.
Où	• `Scripts/Hub/Board.cs:104` : l'appel instancié, par couleur • `Scripts/Hub/Board.cs` : `HasOwnModel` et `Miniature` pour les boss et dirigeables • `Scripts/Core/MeshBatch.cs` : `SmallBall`, la boule de 80 faces
Comment vérifier	• Lancer une vague, revenir au hub, ouvrir le **Frame Debugger** : chercher un appel instancié avec beaucoup d'instances (une par ballon de cette couleur). • **Stats** : le nombre de Batches bouge peu quand le nombre de ballons monte. • Pendant une vague de boss : le boss est bien son modèle sur le plateau, pas une boule étirée.
Ce qu'on perd	les ballons normaux sont des boules simples sur le plateau (ils sont tout petits).

10. Pool de ballons #153, #174

Ce qui a été fait	Un ballon éclaté ou sorti n'est plus détruit : il est caché et rangé, puis ressorti pour le ballon suivant de la même sorte. Créer et détruire des centaines d'objets par vague donnait des à-coups. Depuis le 10 nov., chaque ballon a un **numéro de vie** qui change à chaque sortie : un tir encore en vol vers un ballon éclaté ne suit plus ce ballon quand il ressort à l'entrée de la carte.
Où	• `Scripts/Map/WaveSpawner.cs:35` : le pool (une pile par sorte de ballon) ; `Recycle` ligne 220 • `Scripts/Map/Balloon.cs:186` : `Release` (le ballon retourne au pool) • `Scripts/Map/Balloon.cs:64-67` : `Life` et `StillThere` ; utilisés par `Projectile.cs:91, 107` et `Tower.cs:65` • compatible coop (le double réseau est retiré au rangement) et camo (un ballon recyclé n'est plus camouflé)
Comment vérifier	• En Play, pendant une vague, regarder la Hierarchy de la scène Labyrinthe : les ballons éclatés deviennent grisés (inactifs) au lieu de disparaître ; leur nombre ne grimpe pas sans fin. • **Profiler** → **CPU** : la colonne GC Alloc reste basse pendant une vague. • Poser plusieurs singes sur le même passage, vague serrée : aucune fléchette ne part vers l'entrée de la carte.

11. Rigidbody kinematic sur les ballons #174

Ce qui a été fait	Chaque ballon avait un collider **sans Rigidbody**. Pour la physique, c'est un mur « statique » : le déplacer à chaque image l'oblige à reconstruire sa structure, des centaines de fois par image. Avec un Rigidbody **kinematic**, on le déplace nous-mêmes et la physique le suit sans tout recalculer.
Où	• `Scripts/Map/WaveSpawner.cs:187` : `body.isKinematic = true` (et `useGravity = false`) à la création du ballon
Comment vérifier	• En Play, sélectionner un ballon : composant **Rigidbody**, « Is Kinematic » coché. • **Profiler** → **Physics** : le temps de physique reste faible pendant une grosse vague.

12. Taches du camo en un seul maillage #179	
Ce qui a été fait	Un ballon camouflé avait 6 sphères pour ses taches, donc 6 objets à dessiner en plus. Les taches sont maintenant un seul maillage, partagé par tous les camouflés, sans ombre, et créé une seule fois (le ballon recyclé n'a qu'à le montrer).
Où	• `Scripts/Map/Balloon.cs:256` : `MakeCamo` ; `camoSpots` ligne 55, caché quand le ballon ressort du pool (ligne 97)
Comment vérifier	• Mode admin : aller à la vague 10 ou plus. En Play, sélectionner un ballon camo : il a un seul enfant « Taches » avec un seul Mesh Renderer.

5. Matériaux et ombres

13. Matériaux partagés #153, corrigé le 10 nov.	
Ce qui a été fait	Chaque objet coloré avait sa couleur posée par un « bloc de propriétés », ce qui empêche le SRP Batcher de regrouper les objets : chaque cube coûtait un changement de matériau. Maintenant, **tous les objets d'une même couleur partagent le même matériau**, créé une seule fois. Les couleurs qui changent à chaque image (cadre du tutoriel qui pulse, boutons refusés, panneau d'argent, roulette, nuages) passent par `Animate` et ne créent aucun matériau.
Où	• `Scripts/Core/ColorTint.cs:36` : `Shared`, le cache « un matériau par couleur » • `Scripts/Core/ColorTint.cs:49-50` : `Set` (couleur fixe) et `Animate` (couleur animée) • `Scripts/Core/MonkeyView.cs` (`TintModel`) et `Scripts/Core/BalloonVisuals.cs` (`ApplyTexture`) : même principe pour les singes et les ballons
Comment vérifier	• **Frame Debugger** : les objets de même couleur sont regroupés dans un même « SRP Batch ». • **Stats** : SetPass calls plus bas qu'avant au hub. • Laisser tourner le tutoriel 2-3 minutes, **Profiler** → **Memory** : le nombre de matériaux reste stable (avant la correction, il montait d'environ 72 par seconde).

14. Ombres allégées #153, #177	
Ce qui a été fait	Les ombres s'arrêtent à **15 m**. Les objets petits, nombreux ou rapides ne font plus d'ombre : ballons, taches du camo, projectiles, traits du Sniper, miniatures du plateau, nuages. Chaque ombre redessine l'objet une deuxième fois.
Où	• `Unity/Assets/Settings/Mobile_RPAsset.asset:57` (et `PC_RPAsset`) : `m_ShadowDistance: 15` • `ShadowCastingMode.Off` : `Balloon.cs:139` (modèles), `WaveSpawner.cs:191` (sphères), `Projectile.cs:72`, `Tower.cs:119`, `Board.cs`, `Weather.cs:273`
Comment vérifier	• Sélectionner `Mobile_RPAsset` → Shadows → **Max Distance = 15**. • En Play, sélectionner un ballon ou un projectile : Mesh Renderer → **Cast Shadows = Off**.
Ce qu'on perd	plus d'ombre au sol sous les ballons ; les ombres disparaissent au-delà de 15 m.

6. Décor

15. Décor static et occlusion #153	
Ce qui a été fait	Le décor qui ne bouge jamais est marqué **static** : Unity peut le regrouper et savoir ce qu'il cache. Le calcul d'**occlusion** précalcule, pour chaque endroit, ce qu'on peut voir : depuis la cabane, la bananeraie et la carte derrière les murs ne sont plus dessinées. Les murs de la cabane cachent (Occluder), le reste peut être caché (Occludee).
Où	• `Scripts/Editor/PrototypeGenerator.cs:945` : `MakeStatic` (appelé pour la cabane ligne 886, le décor ligne 963) • `Scripts/Core/MeshBatch.cs:34` : le décor fusionné est marqué static • `Scripts/Editor/OcclusionMenu.cs` : menu **SAE → Calculer l'occlusion (Hub et Labyrinthe)**, les deux scènes ouvertes ensemble
Comment vérifier	• Après chaque **SAE → Générer le prototype**, lancer **SAE → Calculer l'occlusion** (sinon il n'y a pas de données d'occlusion). • Sélectionner un mur de la cabane : la case **Static** (en haut à droite de l'Inspector) est cochée. • Window → Rendering → Occlusion Culling → onglet **Visualization**, caméra dans la cabane : la carte n'est pas dessinée.

7. Singes

16. Singes allégés #153, #182	
Ce qui a été fait	Plusieurs gestes pour que chaque singe coûte moins à chaque image : • **figés** sur le plateau et dans la bibliothèque : leur modèle animé (SkinnedMeshRenderer) est remplacé par un modèle fixe dans la pose de repos, calculé une fois et partagé ; • **immobiles au repos** sur la carte quand personne ne les voit ou à plus de 6 m du joueur (toujours animés quand ils tirent) ; • **ciblage** sans LINQ, et quand aucun ballon n'est à portée, recherche 10 fois par seconde au lieu de chaque image ; • l'**aura** n'est animée que si elle est visible ; au plus 4 projectiles par tir.
Où	• `Scripts/Core/MonkeyView.cs:37-41` : `Freeze` (singes sans `TowerAnimator`) • `Scripts/Map/TowerAnimator.cs:34` : `IdleDistance = 6f` ; le test de visibilité vers la ligne 100 • `Scripts/Map/Tower.cs:21` : `SearchDelay = 0.1f` ; `MaxProjectiles = 4` ligne 12 • `Scripts/Core/Aura.cs:76` : `if (!rend.isVisible) return;`
Comment vérifier	• En Play, au hub : sélectionner un singe du plateau ; son modèle a des **Mesh Renderer**, plus de Skinned Mesh Renderer. • Sur la carte : s'éloigner de plus de 6 m d'un singe au repos, il ne bouge plus ; il se remet à bouger quand il tire. • Mode admin, 200 singes : **Profiler** → **CPU**, le temps des Animator et des scripts baisse par rapport à #153 (41 fps).
Ce qu'on perd	les singes du plateau ne bougent pas (ils font 5 cm).

8. Objets et sons

17. Pools des projectiles, des sons, des ondes et des tirs #178	
Ce qui a été fait	Chaque tir créait un objet et son modèle 3D, détruits à l'arrivée ; chaque son créait un objet et une source audio. Tout est maintenant **réutilisé** : un pool de projectiles par type de singe, un pool d'ondes (Glace, Canon), des traits du Sniper, et **24 sources de son** créées une fois. Les 4 premières sont gardées pour les sons 2D (annonces, voix de Quincy, jingles) : les pops ne peuvent plus les couper ; si tout est occupé, un pop est simplement sauté.
Où	• `Scripts/Map/Projectile.cs:23` : le pool des projectiles • `Scripts/Map/Tower.cs:27` : les traits du Sniper ; `Scripts/Weapon/Shockwave.cs:16` : les ondes • `Scripts/Core/Sounds.cs:72` : `Voices = 24, FlatVoices = 4` ; `FreeVoice` juste en dessous
Comment vérifier	• En Play, Hierarchy (DontDestroyOnLoad) : un objet « Sons » avec **Voix 0 à Voix 23**, toujours les mêmes. • Pendant une vague : les objets « Projectile … » apparaissent puis restent, grisés, et resservent. • Grosse vague + annonce du boss : l'annonce n'est pas coupée par les pops.

18. Préchargement #181

Ce qui a été fait	Le premier pop et le premier boss faisaient un à-coup : chargement du son, création du premier ballon et du premier projectile en pleine partie. Maintenant, tous les sons sont chargés au lancement du jeu (et leur silence de début calculé), et au chargement de la carte on crée d'avance **30 ballons** (dont 10 avec leurs taches de camo) et **12 projectiles** par type de singe posé.
Où	• `Scripts/Core/Sounds.cs:56` : `Preload` (au lancement) • `Scripts/Map/WaveSpawner.cs:200-215` : `Prewarm`, appelé ligne 73 ; `Projectile.Prewarm` (`Projectile.cs:58`)
Comment vérifier	• Dès que la carte est chargée, avant toute vague : la Hierarchy contient 30 ballons inactifs. • **Profiler**, enregistrer pendant le premier pop de la partie : pas de pic de temps CPU.

19. Mises à jour inutiles coupées #182

Ce qui a été fait	Le composant de couleur (`ColorTint`) tournait à chaque image sur des centaines d'objets, alors que ça ne sert qu'à l'arc-en-ciel. En jeu, il est maintenant **désactivé** quand l'objet n'est pas arc-en-ciel : Unity ne l'appelle plus. Changer la couleur marche toujours (et le rallume si on passe en arc-en-ciel).
Où	• `Scripts/Core/ColorTint.cs:71-73` : `SleepUnlessRainbow`
Comment vérifier	• En Play, sélectionner un ballon : le composant **Color Tint** est décoché (sauf un singe arc-en-ciel). • **Profiler** → **CPU** : moins de temps dans *BehaviourUpdate*.

9. Corrections après relecture (10 nov.)

Avant le merge, la PR #188 a été relue en entier. Ce qui a été trouvé et corrigé (commit `07dca58`) :

Problème trouvé	Correction	Où
Un **matériau créé à chaque image** pour les couleurs animées (tutoriel, panneau d'argent, boutons, roulette, nuages) : environ 72 par seconde, jamais libérés. L'inverse du but.	Nouvelle méthode `ColorTint.Animate` (pas de matériau) pour les 11 appels animés ; `Set` ne fait rien si la couleur ne change pas.	`ColorTint.cs` et 11 scripts (`TutorialBoard`, `MoneyBoard`, boutons, roulette, `Weather`)
Les **projectiles suivaient un ballon recyclé** : le ballon éclaté ressort en premier du pool, à l'entrée de la carte, et le tir encore en vol partait vers lui.	Numéro de vie `Balloon.Life`, vérifié à l'arrivée du tir et avant le tir.	`Balloon.cs`, `Projectile.cs`, `Tower.cs`
Sons longs coupés par les pops pendant une grosse vague (annonce du boss, voix, jingle).	4 sources gardées pour les sons 2D ; un pop en trop est sauté.	`Sounds.cs`
Boss en boules étirées sur le plateau (perte visuelle de l'instancing).	Vrai modèle pour les boss et les dirigeables.	`Board.cs`
Le menu des textures n'avait **jamais été lancé**, et son commentaire sur les mipmaps était faux.	Commentaire corrigé, menu lancé (fiche 7).	`TextureMenu.cs`, 39 `.meta`, 18 PNG
L'**occlusion n'avait jamais été calculée**.	Étape ajoutée au README : *SAE → Calculer l'occlusion* après chaque génération.	`README.md`

Pour les retrouver : `git show 07dca58`, ou la PR #188 sur GitHub (section « Corrections après relecture »).

10. Pas fait, et pourquoi

Idée	Pourquoi on ne l'a pas faite
Fusionner les 2-3 maillages de chaque singe en un seul (Blender)	Réexporter des modèles riggés sans pouvoir tester dans Unity, à 4 jours du rendu, c'est trop risqué (noms d'os, échelle, animations). À faire seulement si les mesures montrent que les skinned meshes restent le problème.
Réduire les polygones des ballons dans Blender (~300 triangles)	Remplacé par le Mesh LOD automatique (fiche 6), sans réexport et réversible.
Lumières du hub	Changerait l'image : à décider au casque, pas à l'aveugle.
Managed Stripping « Medium »	Risque de casser XR ou Netcode dans le build.
Alléger la synchro réseau des ballons (#185)	Aucun gain : la rotation d'un ballon normal n'est déjà pas envoyée.
Baisser l'anticrénelage (MSAA 4x)	Le crénelage se voit beaucoup en VR.
LOD Group sur les singes	N'apporterait presque rien sur des singes de 700 triangles.

11. Mesures à faire au casque

Procédure (la même à chaque fois) : build de **développement** sur le casque, compteur de fps au poignet gauche, on regarde dans la même direction 10 secondes et on note la valeur la plus basse. Reporter les résultats dans `docs/opti.md` et dans l'issue #35.

Mesure	Où	Avant (#153)	Après	72 fps ?
fps au hub	face à la bibliothèque			
fps sur la carte	point d'arrivée, vague normale			
fps pendant un boss	carte, orage et pluie			
fps mode admin	200 singes, grosse vague (éditeur)	41 fps		
SetPass calls	Stats, hub (éditeur)	141		
Triangles	Stats, mode admin (éditeur)	626 000		
Matériaux	Profiler Memory, tutoriel 3 min	+72 / s		
Taille de l'APK	fichier du build			

Si une mesure est sous 72 fps : ouvrir le Profiler sur cette scène, regarder si c'est le CPU ou le GPU qui limite, puis ouvrir une issue avec le chiffre et la cause. Ne pas conclure sans chiffre : c'est la règle de `docs/opti.md`.